Distilled Command Parser on a Local Kubernetes Cloud

Cloud Computing (CS3081) Mini Project Report

SIVA NAGA SAI — Roll No. 230103002 — https://github.com/ns998527-rgb/cloud-command-parser

How to read this report. *Measured* means the number comes from a file in `results/` produced by a real run (file named in brackets). *Assumed* means I did not measure it. All timing numbers come from one laptop (Intel i3-1115G4, 4 threads, 7.7 GB RAM, no GPU, Windows 11) running Docker Desktop with 3.65 GiB for the k3d node. The only step done elsewhere is teacher inference (Kaggle, see Task 1). Commit tagged `final`.

Task 1 — Object storage and parallel teacher labelling

Architecture. k3d cluster (`cluster/k3d.yaml`, one server node) → SeaweedFS (master, volume, filer and S3 gateway in one pod, 5 Gi PVC) in namespace `data` with buckets `datasets` and `models` → a worker image (`worker/Dockerfile`) that reads the pool from S3, calls the teacher and writes one shard object per worker → `worker/clean.py` merges the shards. The S3 key pair is generated by `scripts/up.sh` at deploy time and stored only as Kubernetes Secrets, never in Git. [`results/object_storage_listing.txt`] shows both buckets with `commands_pool.jsonl`, `teacher/shard-0.jsonl`, `teacher/shard-1.jsonl`, `v1/train.jsonl` and `v1/train.sha256`.

Teacher and prompt. Phi-3.5-mini (`phi3.5`, Ollama 0.40.15, 2.2 GB), temperature 0, seed 42, `num_predict` 120. `prompts/teacher_prompt.txt` states the schema, the eight intents and the three slot rules, and gives eight worked examples (taken from the brief, not from the validation set). Replies are stored exactly as returned (`teacher_raw`), never edited.

Deviation (measured fact, not hidden). Labelling 3,000 commands on this CPU laptop was estimated at several hours and would not fit next to the cluster in 7.7 GB of RAM. I therefore ran the *same* `worker/worker.py`, prompt and settings on Kaggle's free GPU (`kaggle/label_on_kaggle.py`): about 0.5 s per command, 3,000 + 100 validation commands in roughly 25 minutes. The raw shards were copied into the repository and uploaded to the `datasets` bucket by `scripts/seed_storage.py`. The in-cluster pipeline was run only as a short demonstration (next paragraph).

In-cluster workers (measured, demonstration only). In namespace `pipeline`, an Indexed Job (`k8s/pipeline/teacher-workers.yaml`, image `teacher-worker:1.0`) ran two pods at the same time with `SHARD_INDEX` 0 and 1 and `SHARD_COUNT` 2. Their teacher was Ollama 0.40.15 with `phi3.5` (same digest `61819fb370a3` as on Kaggle) running on the laptop CPU, reached from the pods at `host.docker.internal:11434`: an Ollama *pod* needs about 3 GB, which the 3.65 GiB node cannot spare next to the application, and the brief allows the teacher on the laptop. To keep it short, `LIMIT=5` made each worker label only its first 5 commands. Each worker wrote its own object, `datasets/teacher-incluster/shard-0.jsonl` and `shard-1.jsonl` [`results/pipeline_demo/`]. The full 3,000 labels in `datasets/teacher/` are the Kaggle ones.

Reproducibility finding (measured; cause not proven). For the same 10 commands the laptop replies were byte-identical to Kaggle's in only **1 of 10** cases, although model digest, Ollama version, prompt, temperature 0 and seed 42 were the same. The laptop replies often kept generating after the JSON ('

Command: …), and 3 of 10 differed in an intent or slot (e.g. C0003calendar_setagainstcalendar_query) [results/pipeline_demo/comparison_with_kaggle.txt]. A follow-up test shows the laptop is self-consistent: 5 commands sent twice, one at a time, gave identical replies both times

and identical to the earlier in-cluster run where two workers queried Ollama at once [results/pipeline_demo/laptop_determinism_test.txt]. So request order and concurrency are not the cause; the difference lies between the two environments. I did not isolate it; my assumption is that GPU and CPU back-ends round differently, which can flip near-ties in greedy decoding. Either way the raw replies and the checksum of the cleaned file are what I treat as the fixed input, andclean.py' rejects replies with trailing text under rule (a).

Sharding (explanation). Worker `i` of `SHARD_COUNT = 2` takes exactly the commands whose 0-based line number `n` satisfies `n mod 2 = i`, so shard 0 gets the even lines and shard 1 the odd lines, 1,500 commands each. The rule needs no coordination, is deterministic and has no gaps or overlaps by construction. I checked it on the real output: the 3,000 ids across both shard files are unique and equal to the pool ids (every id in exactly one shard). Each worker writes its own object (`shard-N.jsonl`) and flushes every 25 records, so a restarted pod resumes instead of starting over. `clean.py` sorts by id, so the result and its checksum do not depend on the number of shards.

Cleaning results (measured, `results/clean_counts.json`). Rejections are counted once, under the first rule that fires; `input = (a)+(b)+(c)+(d)+duplicates+kept`.

Input	(a) invalid JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	Duplicates	**Kept**
3,000	310	0	5	525	31	**2,129**

Check: 3,000 = 310 + 0 + 5 + 525 + 31 + 2,129. The SHA-256 in `datasets/v1/train.sha256` is `691d957871794e23b8b3dd4982544dfef24073c8e734323470d1eab01896486b`. The largest rejection group (17.5%) is (d): the teacher often wrote the *string* `"null"` for a missing slot, or paraphrased a slot (e.g. `7:30`), which rule (d) correctly rejects.

Limitations. Teacher labels are noisy (the teacher itself scores only 36% full-frame on validation). One SeaweedFS pod with a local-path volume has no replication. The in-cluster Job labelled only 10 commands, not the whole set. **Improvements.** Run Ollama on a larger node and execute the Job for the full set; use constrained (JSON-grammar) decoding to avoid invalid replies; add a retry-on-invalid step in the worker.

Task 2 — Training the student (sequence-level distillation)

Setup. `training/config.yaml` records the seed (42), learning rate (1e-4), batch size (16), epochs (5), maximum sequence length (128) and library versions (torch 2.4.1, transformers 4.46.3, pyyaml 6.0.2, numpy 1.26.4). `make train` reads only that file; `make train SUBSET=50` trains on the first 50 records (verified to run to the end: 2 epochs on a throw-away file took 27 s). Each example is `Command: <command>\nJSON: <teacher reply><eos>`; the loss is computed on the JSON part only. EleutherAI/pythia-70m is fine-tuned in full on the first 1,500 records of `train.jsonl` on CPU, AdamW with linear warm-up and decay. v1: seed 42, 5 epochs, 1,915 s. v2 (required for Task 4): seed 7, 6 epochs, 6,950 s (slower because other jobs shared the CPU). Both models and tokenizers are in `models/v1` and `models/v2` in the bucket and are GitHub Release assets (`model-v1`, `model-v2`); `models/<v>/model.sha256` (in Git) lists the SHA-256 of every file and the CI job re-downloads and verifies them.

Epoch	1	2	3	4	5	6
v1 train loss [`training/loss.csv`]	0.4160	0.1302	0.0750	0.0457	0.0234	–
v2 train loss [`training/loss_v2.csv`]	0.4171	0.1451	0.0809	0.0524	0.0301	0.0154

Evaluation on `validation.jsonl` (measured, 100 commands; raw outputs and reports in `results/`).

Model	json_validity	intent	date	time	place	full_frame
Teacher phi3.5 (same prompt/settings)	92	74	60	76	85	36
Base pythia-70m (untrained)	0	0	0	0	0	0
Student v1	**100**	78	68	87	88	**40**
Student v2	100	81	69	89	86	40

What changed. The untrained model never produces valid JSON (e.g. `- name: "calendar"`), so fine-tuning is what creates the format. The student reaches 100% valid JSON, above the teacher's 92%, because it was trained only on replies that passed `clean.py`. Its slightly higher full-frame score (40 vs 36) is **within noise**: with 100 examples the standard error of a proportion near 0.4 is about 5 percentage points, so I do not claim the student is better than the teacher. Remaining errors mostly follow the gold labels' conventions (the labels come from SLURP, not from the teacher): e.g. for "five in the morning" the gold time is `five` while the student copies the longer phrase. Suitability of pythia-70m: it has no instruction-following ability, but 1,500 examples are enough to learn this narrow, rigid format; its capacity limits semantic accuracy. **Assumption (not measured):** the hidden 300-command set behaves like validation, i.e. validity ≥ 95% and full-frame above half of the teacher's (18%).

Sequence-level vs logit-based distillation (explanation). (a) *Sequence-level* distillation trains the student with ordinary cross-entropy on the teacher's generated output as a hard target: the student sees the text the teacher wrote, nothing else. That is what I did. (b) *Logit-based* distillation trains the student to match the teacher's whole probability distribution over the vocabulary at every token (soft targets, typically a KL-divergence loss), which carries "dark knowledge" about how close the alternatives were. (c) It is not possible here because the teacher was only used as a text generator through Ollama, so I never obtained its logits; and Phi-3.5 and Pythia use different tokenizers and vocabularies, so their per-token distributions cannot be matched position by position without extra alignment work. (d) One real limitation of my approach: the student copies the teacher's single greedy answer, mistakes included, with no information about the teacher's uncertainty. Roughly one in six teacher records had to be discarded for an unsupported slot, and wrong-but-valid replies remain in the data, which caps the student near the teacher's quality (teacher intent accuracy 74%).

Task 3 — Inference service in a container

Architecture and implementation. `service/app.py` (FastAPI + uvicorn, port 8000). `POST /parse` returns the four-key JSON with headers `X-Model-Version` (`MODEL_VERSION`) and `X-Served-By` (the Pod name). `GET /healthz` is 200 while the process lives; `GET /readyz` is 503 until the model is loaded (the load runs in a background thread, followed by one warm-up generation), then 200. Configuration comes only from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS` (plus `FAULT_INJECT`); credentials come from a Kubernetes Secret. At start-up the pod downloads every object under `MODEL_URI` from SeaweedFS (log: `downloaded 8 files from s3://models/v1/`). Every model output is checked against `schema.json`; an invalid output gives HTTP 500 with the body `{"error": "invalid_model_output"}`, never broken JSON with 200. The image (`Dockerfile`, `python:3.11.10-slim`) installs CPU-only `torch==2.4.1+cpu` and pinned requirements, contains no weights and builds with one command (`docker build -t command-parser:1.0 .`). **Measured:** image size 1.7 GB on disk (0.37 GB compressed), under the 2 GB limit.

Results (measured). Fault injection [`results/task3_fault_inject.txt`]: with `FAULT_INJECT=1`, `/parse` returned `HTTP 500` and exactly `{"error":"invalid_model_output"}`. Memory test [`results/task3_50_requests.txt`]: 50 sequential requests to one pod limited to 1 CPU and 1 GiB all returned 200 (mean 0.82 s, median 0.56 s, max 3.58 s); no restart, no OOM kill; **peak memory 757,985,280 bytes (723 MiB)** from the container's own `memory.peak`. Eight unit tests pass (`make test`).

Limitations. A lock serialises generation inside each pod. fp32 weights and a single torch thread keep memory low but make each request CPU-bound. The model downloads on every cold start. **Improvements.** int8 dynamic quantization (I measured 1.69× faster generation on 30 validation commands, but 10 of the 30 outputs changed, so I did not ship it), request batching, caching the weights on a shared volume.

Task 4 — Kubernetes deployment, scaling and reliability

Architecture. Namespace app: Deployment `parser` (2 replicas, image `command-parser:1.0` imported into k3d, requests 250m CPU / 512 Mi, limits 1 CPU / 1 Gi), Service (80 → 8000), Ingress (Traefik, `/parse` → Service, host port 8080) and HPA (min 1, max 2, CPU 70%). Startup probe on `/healthz`, liveness on `/healthz`, readiness on `/readyz`. Rolling update with `maxUnavailable: 0`, `maxSurge: 1`. All through `make up` (`scripts/up.sh`), one command from a fresh clone.

An honest failure first. My first rolling-update run [`results/attempt1_*`] was invalid: **14,071 of 14,328 requests failed**. Cause (from `kubectl describe`/logs): Traefik, CoreDNS and metrics-server had been restarted 10+ times, because their own health probes timed out while Windows memory pressure starved the node (host free RAM 0.4–0.7 GB, compressed memory 2 GB); every Traefik restart made port 8080 refuse connections. Fix: `scripts/harden_cluster.sh` gives those system pods long probe timeouts (10 s, 10 failures), and I relaxed the parser's own probes. The numbers below are from runs after the fix; the failed run is kept as evidence.

Experiment (all measured)	Result	Evidence (in `results/`)
Rolling update v1→v2 under load, then `kubectl rollout undo`, then pod kill, in one 9-minute Locust run	862 requests, **2 failed (0.23%)**, both HTTP 502 near the switch to v2; served by v1: 690, v2: 173	`rolling_update_log.txt`, `rolling_load_*`
Rollback	three probes after undo returned `X-Model-Version: v1`; Deployment env back to v1	`rolling_update_log.txt`
Recovery (delete a Pod under load)	ready replicas 2 → 1 → **2 again in about 39 s** (04:47:07 → 04:47:46)	`recovery_log.txt`
Cold start (delete all Pods ×3)	creation→Ready **140 s, 53 s, 54 s; median 54 s**; logs show `downloaded 8 files from s3://models/v1/` and `model ready in 38–125 s`	`coldstart*.txt`
No ready endpoints while loading	20 consecutive samples with an empty ready-address list (Pods listed as not-ready); Ingress answered 503; one ready endpoint appeared only after the model loaded	`endpoints_while_loading.txt`
HPA 1→2→1	idle at 1 replica; load starts 04:58:55; CPU 187% → 396% of target; both replicas Ready by 05:00:20; load stops 05:03:31; back to 1 replica at 05:08:48. During this run 434 requests, 0 failures, p95 4.5 s	`hpa_scaling.txt`, `hpa_load_*`

Experiment (all measured)	Result	Evidence (in `results/`)
Load balancing, exactly 2 Ready replicas, standard command through the Ingress	replica A 112, replica B 114 successful requests (49.6% / 50.4%)	`loadtest_headers.json`

Limitations. One laptop, one k3d node: this shows scheduling, self-healing and rolling updates but **not** fault tolerance across machines or zones; if the node or Docker dies, everything is down. The HPA percentage is relative to the 250m CPU *request*, so a busy pod (about 1 core) reads 370% and scaling is eager; metrics-server briefly returned "unknown" while the node was starved; scale-down waits the default 5-minute window. The 3.65 GiB node leaves little room for a surge Pod. **Improvements.** Multi-node cluster, PodDisruptionBudget, requests set from measured use (about 900m), and more memory for the Docker VM.

Task 5 — Subnetting and network access control

Method: allocate from the largest to the smallest from 10.20.0.0, using the smallest prefix whose `2^(32−prefix) − 2` hosts fit (details and working in `network/subnets.md`; verified with Python's `ipaddress`).

Subnet	Network	Prefix	Usable range	Broadcast	Usable hosts
app (1,000)	10.20.0.0	/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public (250)	10.20.4.0	/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data (100)	10.20.5.0	/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management (20)	10.20.5.128	/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

NACL for the public subnet (`network/nacl.json`). Inbound: rule 90 deny all from 198.51.100.0/24; 100 allow TCP 443 from 0.0.0.0/0; 110 allow TCP 80 from 0.0.0.0/0; 120 allow TCP 22 from 10.20.5.128/27 (management); 130 allow TCP 1024–65535 from 10.20.0.0/22 (replies from app). Outbound: 100 allow TCP 8000 to 10.20.0.0/22 (app); 110 allow TCP 1024–65535 to 0.0.0.0/0 (replies to clients). Everything else is denied by the implicit final deny. **Measured:** `nacl_sim.py --check` on the six sample packets gives 6/6 [`results/nacl_check.txt`]. The hidden 20-packet score is not known (assumption that the rules follow the brief literally).

Why replies need their own rules; why order matters. A NACL is stateless: it does not remember that a request was allowed, so a reply is judged as a brand-new packet. A client connects to port 443 from an ephemeral port in 1024–65535; inbound rule 100 admits the request, but the response leaves as a new outbound packet to that ephemeral port, which only outbound rule 110 allows (and likewise the load balancer's call to the app on 8000 is answered inbound to an ephemeral port, admitted by rule 130). Rules run from the lowest number up and the first match wins, so the deny for 198.51.100.0/24 must be numbered below the allow rules for ports 443 and 80: a packet from 198.51.100.9 to port 443 then hits rule 90 first and is dropped. Numbered above 100, it would match the allow-443 rule first and be let in.

Task 6 — Reproducibility, testing and CI/CD

Implementation. `make up` builds and deploys everything (cluster, secrets, SeaweedFS, buckets and uploads with models fetched from the GitHub Release and checksum-verified, images, application); `make down`

deletes the cluster. `.github/workflows/ci.yml` has the required stages in order: `lint` (ruff), `unit-tests` (pytest, 8 tests), `evaluate` (downloads the model from the GitHub Release, verifies SHA-256, runs `training/predict.py` and `tools/score.py` on `validation.jsonl`) and `quality-gate` (`score.py --min-validity 95`), plus a `secret-scan` job (gitleaks) that reports nothing. A manual `workflow_dispatch` input selects the model.

Fresh-clone check (measured). After `make down` (10 s, cluster deleted) I cloned the repository from GitHub into an empty folder and ran `make up`: it completed in **18 min 55 s** (this includes downloading and checksum-verifying both models from the Releases and building both images) and a request through the Ingress returned the correct JSON [`results/fresh_clone_make_up.log`, `results/make_down.log`]. On Windows I set `PYTHON` to an interpreter that has `boto3`; the prerequisites are Docker, k3d, kubectl, make and `pip install boto3`.

Evidence (measured). Passing run with the v1 Release asset (all five jobs green): https://github.com/ns998527-rgb/cloud-command-parser/actions. Deliberately failing run with the untrained pythia-70m (`model=base`); lint, tests and evaluation pass but the **quality gate fails** because its JSON validity is 0%: https://github.com/ns998527-rgb/cloud-command-parser/actions. `AI_USE.md` lists the AI tools used, for what, and how I checked the output.

Limitations and improvements. The evaluation job uses CPU runners and takes a few minutes; the CI never deploys to Kubernetes (no cluster in CI). A kind/k3d cluster in the workflow and caching of the PyTorch wheel would make it a full CD pipeline.

Task 7 — Performance, cost and review

Load test (measured). Exact standard command (4 users, 2 min, through the Ingress, two Ready replicas) [`results/loadtest_*`, `task7_kubectl_top.txt`]:

Measure	Value
Requests / failures (error rate)	225 / 0 (**0.00%**)
p50 / p95 latency	1,900 ms / **4,000 ms**
Throughput	**1.91 requests/s**
`kubectl top pods` at 30 / 60 / 90 s	338m, 162m (594, 571 Mi) / 997m, 974m (537, 549 Mi) / 891m, 898m (526, 538 Mi)
Cold start (Task 4)	median 54 s (140, 53, 54 s)

Cost (estimate from `rate_card.json`; no real charge). Each replica is billed for its limits: 1 vCPU × ₹3.50 + 1 GiB × ₹0.45 = ₹3.95 per hour. Two replicas: 2 × 3.95 = **₹7.90 per hour**. Requests per hour = 1.91 × 3,600 = 6,876. Cost per 1,000 requests = 7.90 / 6,876 × 1,000 = **₹1.15** (1.149). (Counting only the two parser replicas; SeaweedFS, whose limits are 1 CPU / 768 Mi, is excluded as it is not a service replica. The figure rests on the assumption that throughput stays at the measured 1.91 req/s for a full hour.)

SLO. Target: 99% of `/parse` requests return HTTP 200 within 30 s, window = one standard load-test run (225 requests). Error budget = 1% × 225 = 2.25, i.e. 2 failed requests. Used: 0 of 2 (0% of the budget). Additional latency objective: p95 ≤ 5,000 ms; measured 4,000 ms (met, with 1 s margin).

Review table (each evidence cell is a measured number).

Component	Evidence	Limitation found	Improvement
Data and storage	3,000 → 2,129 kept; 525 (17.5%) dropped for slots not in the command	Noisy teacher labels; single SeaweedFS pod, no replication	Constrained JSON decoding; retry on invalid; replicated volumes
Model quality	student v1 full-frame 40%, intent 78%, validity 100% (teacher 36 / 74 / 92)	100-example validation set (±5 pp); student copies teacher errors	More data and epochs; larger student; logit-based distillation
Latency and throughput	p50 1.9 s, p95 4.0 s, 1.91 req/s	fp32, one thread, serial generation per pod: CPU-bound	int8 quantization (1.69× faster, but 10/30 outputs changed); batching
Resource utilisation	pods about 0.9–1.0 CPU and 526–594 MiB under load; 723 MiB peak (cgroup)	Requests (250m) far below real use, so HPA reads 370%; node has only 3.65 GiB	Set requests from measurements; bigger node
Reliability and recovery	update + rollback + pod kill: 0.23% failed; replica back in about 39 s; cold start median 54 s	One node: no node-failure tolerance; first run lost 14,071 of 14,328 requests when starved Traefik restarted	Multi-node, PodDisruptionBudget, tolerant probes (done)
Security and cost	₹1.15 per 1,000 requests; gitleaks run reports nothing; no secrets in Git	One S3 admin key shared by all namespaces; no TLS or NetworkPolicy; open Ingress	Per-service credentials, TLS, NetworkPolicies, image scanning

Reflection (short) and references

Main lessons: distillation is cheap when the format is narrow; most of the engineering effort went into making a heavy workload survive a small machine (probes, memory, honest measurement). Things I would do differently: start with more node memory, and run the whole teacher Job in the cluster.

References. Kim & Rush, *Sequence-Level Knowledge Distillation*, EMNLP 2016. Bastianelli et al., *SLURP*, EMNLP 2020 (CC BY 4.0). EleutherAI pythia-70m model card. Kubernetes, k3d, SeaweedFS, Traefik, Locust and gitleaks documentation. Phi-3.5-mini via Ollama.